

Design of a Low-Cost, Low-Power Embedded Device for IMU Motion Data Collection in OpenSim

Nathan Jones, Syed Razwanul Haque

Oregon State University

Abstract. Optical motion capture systems are costly and bound to the laboratory setting, limiting access to human movement analysis in portable and field-based settings. This paper presents a low-cost, low-power embedded device for OpenSim-compatible IMU data collection, employing an ESP32-C6 microcontroller, BNO055 IMU sensors, a TCA9548A I2C multiplexer, SD card storage, and an onboard Wi-Fi server for wireless file retrieval. The system records quaternion orientation data directly in OpenSim-compatible format without requiring a computer connection. When tested with four IMU sensors, the device operated at 70-100 Hz, consumed 0.30-0.32 W, and weighed 105 g, which is approximately one-tenth the power draw of the state-of-the-art Raspberry Pi based setup priorly demonstrated in the literature. The system was validated by being deployed on the upper-limb, successfully reconstructing elbow and shoulder flexion in OpenSim using the IMU Placer and Inverse Kinematics pipelines. The design offers a reproducible, low-barrier platform for IMU-based motion analysis outside the laboratory.

1 Introduction

Motion analysis plays an important role in human and animal biomechanics, rehabilitation, sports science, robotics, and movement studies. Traditional optical motion capture systems can provide accurate kinematic measurements, but they are expensive, require controlled laboratory environments, and usually depend on cameras, markers, and careful calibration. These requirements limit their use in low-cost, portable, and field-based applications. Inertial measurement units (IMUs) provide a practical alternative because they are small, low-cost, wearable, and capable of estimating body segment orientation without requiring external cameras.

OpenSim is widely used for biomechanical modeling and motion visualization. Its OpenSense workflow allows IMU orientation data to be used for musculoskeletal motion analysis. However, many OpenSense-compatible systems rely on relatively power-hungry comput-

ing platforms or real-time streaming setups. For long-duration recording, field experiments, and animal or farm-based motion monitoring, a smaller and lower-power embedded system is more suitable. Such a system should be able to collect data from multiple IMUs, reliably store orientation data, and provide an easy way to access the recorded files after data collection.

This project presents a low-cost and low-power embedded IMU data collection device for OpenSim-compatible motion visualization. The system is built using an ESP32-C6 microcontroller, multiple IMU sensors, a TCA9548A I2C multiplexer, and an SD card module. The ESP32-C6 platform was selected because it provides a compact embedded controller with RISC-V processing capability and low-power operation suitable for wearable sensing applications [4]. Unlike Raspberry Pi based systems, the proposed device does not require a traditional operating system, which reduces software overhead and simplifies deployment. Since the system is developed using widely available microcontroller and sensor modules, it is easy to assemble, replicate, and maintain. The device records quaternion orientation data from multiple IMUs and saves the data directly to an SD card in OpenSim-compatible format. In the current implementation, four IMUs are used to demonstrate upper-arm and forearm motion visualization in OpenSim, although the hardware design can be extended to support additional sensors.

A key practical feature of the proposed system is its onboard file access capability. After data recording is completed, the ESP32-C6 can create a local Wi-Fi access point and host a lightweight web server. Users can connect to the device through a standard web browser and download the recorded OpenSim-compatible files directly from the SD card. This removes the need

to physically remove the SD card after each experiment and improves convenience for repeated trials, portable experiments, and field-based data collection.

The device was tested with four IMU sensors operating at approximately 70 to 100 Hz. At this sampling range, the system consumed an average power of approximately 0.30 to 0.32 W during full-speed operation. The measured total weight of the complete device, including four IMU sensors, was approximately 105 g, making the system lightweight and suitable for wearable and portable motion capture applications. Therefore, the proposed ESP32-C6 based system provides a more energy-efficient solution for OpenSim-compatible IMU data logging, consuming approximately 4.7 to 10 times less power than the Raspberry Pi based reference setup.

The long-term motivation of this work is to support portable and low-power motion capture outside the laboratory. While the current demonstration focuses on human upper-limb motion, the same design can be extended to animal and farm-based movement monitoring. For example, lightweight IMU nodes could be attached to animal limbs or body segments to collect motion data over longer periods without relying on camera-based tracking. This potentially makes the proposed system suitable for future studies involving wearable sensing, animal biomechanics, and field-based motion analysis.

The main contributions of this project are:

- a low-cost, low-power, and lightweight embedded IMU data logger;
- direct SD card logging of OpenSim-compatible quaternion data;
- a multi-IMU architecture using an ESP32-C6 and TCA9548A multiplexer;
- browser-based file download through a local Wi-Fi web server;
- power, weight, endurance, cost, and maintainability comparison with an OpenSenseRT-style setup.

2 Related Work

IMU-based motion capture has become an important alternative to camera-based motion cap-

ture because IMUs are wearable, portable, and relatively low-cost. OpenSense is one of the most relevant tools for this project. Al Borno et al. developed OpenSense as an open-source framework for estimating joint kinematics from IMU data using OpenSim and sensor fusion methods [1]. OpenSense provides the foundation for using IMU orientation data with OpenSim inverse kinematics.

OpenSenseRT is also closely related to this work. Slade et al. developed OpenSenseRT, an open-source wearable system for real-time measurement of three-dimensional human motion using IMUs [2]. Their work demonstrated that wearable IMU systems can estimate upper- and lower-body kinematics. However, the OpenSenseRT-style setup is more complex and hardware-intensive for beginner users because it relies on a central Raspberry Pi-based system and multiple connected components. In addition, compatibility issues have been reported when running older OpenSenseRT code with newer OpenSim versions [3].

Other low-cost IMU motion capture systems have also been proposed. These studies show that affordable embedded IMU systems can reduce the barrier to human motion analysis. However, many of these systems focus on general motion capture and do not directly generate OpenSim-compatible files. In contrast, the system presented in this work focuses on a simple embedded architecture that directly records OpenSim-ready quaternion `.sto` files for subsequent motion visualization and analysis.

3 Methods

3.1 System Overview

The proposed system is a portable embedded IMU data logger for OpenSim. The main hardware components are an ESP32-C6 microcontroller, BNO055 IMU sensors, a TCA9548A I2C multiplexer, an SD card module, a push button, an LED indicator, and a battery.

Fig. 1 shows the overall system pipeline. The IMUs collect body-segment orientation data. The TCA9548A multiplexer allows multiple IMUs

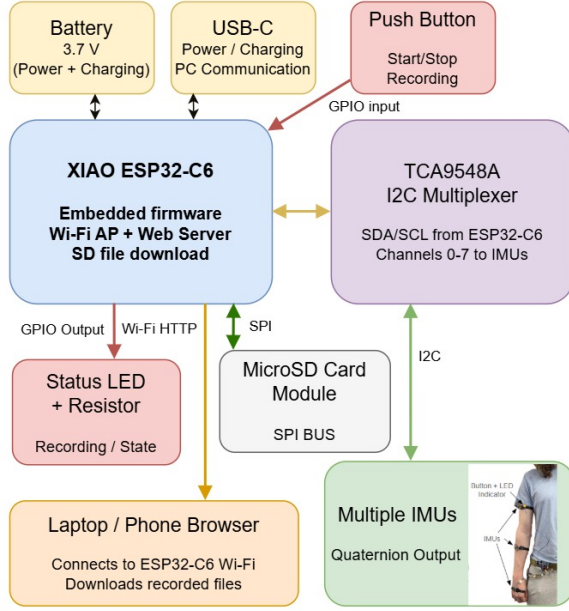


Fig. 1. Block diagram of the proposed embedded IMU data collection and OpenSim processing pipeline.

with the same I2C address to connect to one ESP32-C6. The ESP32-C6 reads quaternion data from each sensor and saves the data to an SD card. The saved `.sto` files are then used in OpenSim for IMU placement and inverse kinematics.

3.2 Embedded Device Design

A prototype of the embedded device is depicted in Fig. 2. The ESP32-C6 was selected because it is small, low-cost, low-power, and compatible with Arduino-based development. The BNO055 IMU was used because it can provide fused quaternion orientation data directly. This reduces the software complexity on the microcontroller. The TCA9548A I2C multiplexer was used to support multiple BNO055 sensors because these sensors can share the same I2C address.

An SD card module was used for local data storage. This allows the system to collect data without a continuous computer connection. A push button controls the recording workflow. A red LED provides feedback to the user. During initial pose recording, the LED blinks rapidly. During motion recording, the LED blinks slowly.

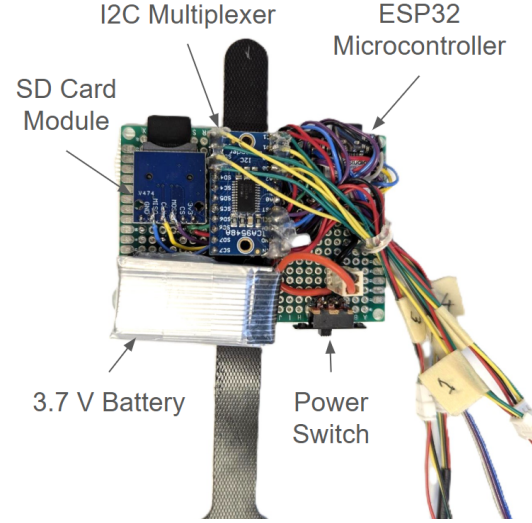


Fig. 2. Prototype of the low-cost embedded IMU data collection device.

3.3 Sensor Placement and Recording Workflow

The system was tested using two and three IMU configurations. For the two-IMU setup, one sensor was placed on the right upper arm and one sensor was placed on the right forearm. For the three-IMU setup, a third sensor was added on the hand. The three IMU setup for motion data collection is shown in Fig. 3.

The three-IMU channel mapping was:

- TCA channel 0: right upper arm, `humerus_r_imu`
- TCA channel 1: right forearm, `radius_r_imu`
- TCA channel 2: right hand, `hand_r_imu`

The sensors were mounted flat on the body segment and strapped tightly. This is important because motion between the IMU and the body segment can introduce error.

The device records two files. The first file is the initial pose file, named `placement_orientations.sto`. This file is recorded for three seconds while the user holds a neutral pose. The second file is the motion file, named `walking_orientations.sto` in the figure, but represents the arm motion data for our system. This file contains the actual movement trial.

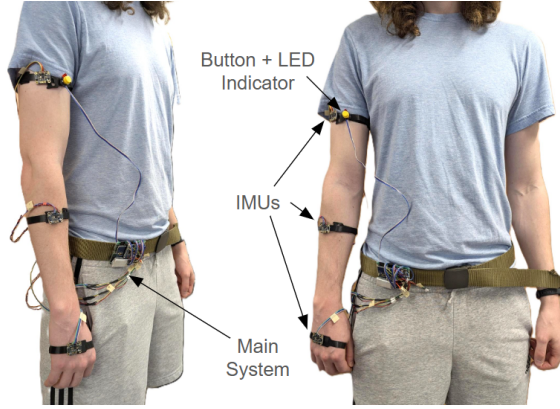


Fig. 3. Example IMU placement on the upper arm, forearm, and hand during data collection.

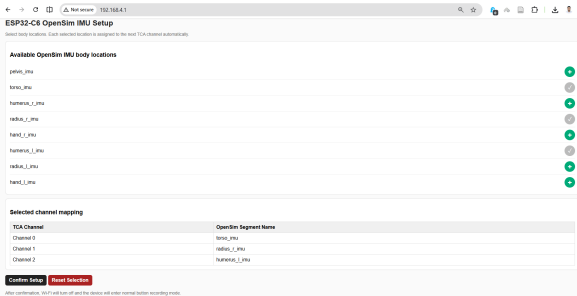


Fig. 4. Bespoke user interface for mapping IMU channels to limb segments and generating OpenSim-ready .sto files.

Both files are saved in OpenSim-compatible quaternion format. The file header contains `DataRate=100.000000`, `DataType=Quaternion`, `version=3`, `OpenSimVersion=4.5`, `endheader`, and the sensor labels `time humerus_r_imu`, `radius_r_imu`, `hand_r_imu`. Each row contains the timestamp and quaternion orientation values for each IMU.

To streamline the process of assigning quaternion data from a given IMU to the correct limb segment, a dynamic web user interface was developed. This interface is depicted in Fig. 4. Before the data are downloaded, the user can manually select the limb segment on which the IMU is located, and the file output is automatically formatted so that it can be directly imported into OpenSim for motion analysis. This feature greatly streamlines the process, reducing the need for users to modify the files directly.

3.4 OpenSim Processing

The recorded IMU data were processed in OpenSim using two main tools: IMU Placer and IMU Inverse Kinematics. The purpose of this processing step was to convert the recorded quaternion orientation data into reconstructed body and joint motion in the OpenSim musculoskeletal model. The OpenSim processing workflow is shown in Fig. 5.

In the first step, the IMU Placer tool was used for model calibration. This step aligns the virtual IMU frames in the OpenSim model with the measured IMU orientations from the initial static pose. The inputs to the IMU Placer step were the base OpenSim model, `Rajagopal2015_opensense.osim`, the initial pose orientation file, `placement_orientations.sto`, and the setup file, `imuPlacer.xml`. The output of this step was a calibrated OpenSim model, `Rajagopal2015_opensense_IMU.osim`.

In the second step, the calibrated model was used with the IMU Inverse Kinematics tool. This step estimates body and joint motion by fitting the OpenSim model to the recorded IMU orientation data over time. The inputs to the IMU Inverse Kinematics step were the calibrated model, `Rajagopal2015_opensense_IMU.osim`, the motion orientation file, which would be the equivalent of `walking_orientations.sto`, and the setup file, `imuInverseKinematics_Setup.xml`. The output of this step was a motion file, equivalent to `walking_orientations.mot`, which contains the reconstructed OpenSim motion.

For the dumbbell-curl-like motion tested in this project, the most important model degrees of freedom were `arm_flex_r` and `elbow_flex_r`. Additional degrees of freedom such as `arm_add_r`, `arm_rot_r`, `pro_sup_r`, `wrist_flex_r`, and `wrist_dev_r` can be enabled depending on the type of movement being tested. However, enabling too many degrees of freedom at once can make the motion harder to interpret. Therefore, simple isolated motions were tested first before using more complex upper-limb movements with more degrees of freedom enabled in the OpenSim simulation.

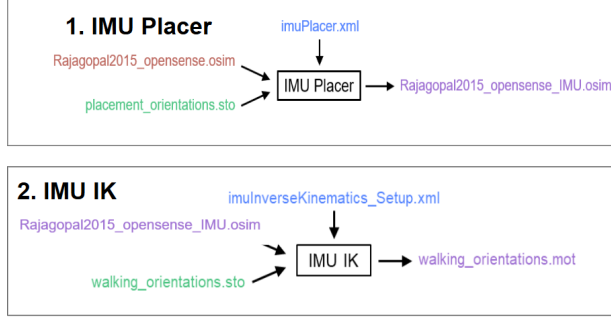


Fig. 5. OpenSim processing workflow using IMU Placer and IMU Inverse Kinematics.

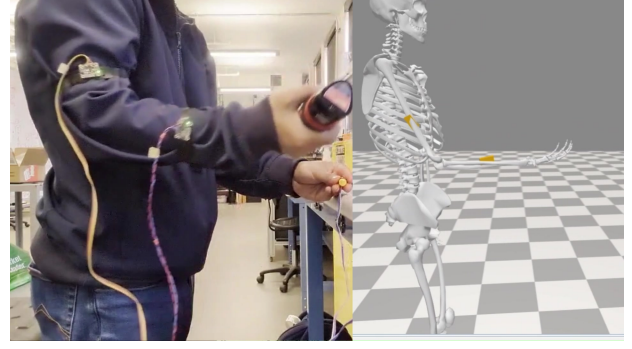


Fig. 6. Side-by-side demonstration of the real arm movement and the reconstructed motion in OpenSim.

4 Results

4.1 Device Operation

The prototype successfully collected quaternion data from multiple IMUs and saved the data to the SD card. The button-controlled workflow produced both the initial pose file and the motion file. The LED feedback system also worked as intended. The recorded files were compatible with OpenSim IMU Placer and IMU Inverse Kinematics.

4.2 OpenSim Motion Demonstration

A side-by-side video demonstration was recorded to compare the real arm movement with the OpenSim reconstructed motion as shown in Fig. 6. The user performed a dumbbell-curl-like movement where the upper arm and forearm moved toward the body and face. The OpenSim skeleton showed similar upper-arm and forearm motion. Some differences remained, in terms of the speed of motion and range of motion, but the general movement direction and joint behavior were similar.

4.3 Power Consumption and Storage

To test power consumption, the device was tested with four IMU sensors during the data collection stage, to test the highest power consumption case of the device. The average power consumption was approximately 0.30-0.32 W while recording data at approximately 70-100 Hz. With this rate of power consumption, the device can run for

around 34 hours continuously with a 3000 mAh battery. This low power consumption makes the device suitable for longer portable and wearable motion data collection.

The storage requirement is also practical for long-duration recording. At 100 Hz, assuming approximately 340 bytes per row for the current four-IMU data format, the storage required per hour is:

$$100 \times 3600 \times 340 = 122,400,000 \text{ bytes/hour}$$

$$122,400,000 \text{ bytes/hour} \approx 122.4 \text{ MB/hour}$$

For an eight-IMU configuration, the estimated storage requirement is approximately 245 MB/hour at 100 Hz. Therefore, an 8 GB SD card can store approximately 32.6 hours of data at 100 Hz, highlighting high synergy between the power and storage capabilities of the device due to their similar deployable lifespan under continuous operation.

4.4 Comparison with OpenSenseRT-style Setup

Table 1 compares the proposed ESP32-C6 device with an OpenSenseRT-style Raspberry Pi setup. The comparison focuses on practical deployment factors such as power consumption, weight, endurance, cost, operating system requirement, and maintainability.

The proposed device is not intended to replace all OpenSenseRT features. Instead, it focuses on low-power local data collection and OpenSim-compatible file generation. Compared with the

Table 1. Comparison between the proposed device and an OpenSenseRT-style device from [2].

Metric	Our Device	OpenSenseRT
Power	~0.30-0.32 W	~1.5-3.0 W
Weight	~105 g	~408 g
Endurance(400mah)	~4.6 h	~0.5-1 h
Endurance(3000 mAh)	~34 h	~3.7-7 h
Cost	~32 USD	~170 USD
OS	Embedded firmware	Linux OS
Real Time IK	No	Optional
Maintainability	Easy	Moderate-difficult
Main Board	Readily available	Very limited
IMU Segment Mapping Interface	User-Selectable	Not Available

OpenSenseRT-style Raspberry Pi setup, the proposed device is lighter, consumes less power, has longer battery endurance, and is easier to maintain because it uses embedded firmware instead of a Linux-based operating system.

5 Discussion

The results show that a low-cost ESP32-C6 based device can collect IMU orientation data for OpenSim. The main advantage of the system is accessibility. The device can be built from low-cost components and does not require a motion capture lab or a continuous computer connection during recording.

Compared with a Raspberry Pi/OpenSenseRT-style system, the proposed device has lower power consumption, lower weight, and simpler hardware. It is also easier to adapt to future microcontrollers because the design is based on standard I2C, SPI, and SD card logging. This makes the device more maintainable for student projects and small research groups.

There are several limitations. The current validation is mostly qualitative and based on visual comparison with the recorded video. The system was not validated against optical motion capture. Sensor placement can also affect the result because the IMU may move relative to the bone due to soft tissue motion. The BNO055 orientation can also be affected by magnetic disturbances. Future work should include better wearable mounting, more sensors, longer power testing, and quantitative comparison with a refer-

ence motion capture system. Additionally, more complex motions with higher degrees of freedom should be demonstrated and validated with the device, since real-world movements are often much more complex than a simple bicep curl.

6 Conclusion

This paper presented a low-cost, low-power embedded device for IMU motion data collection in OpenSim. The device is a self-contained embedded logger that records directly to OpenSim-compatible .sto files without requiring a continuous computer connection. The system records quaternion data and saves it directly in OpenSim-compatible .sto files.

The prototype was tested on the upper-limb, demonstrating upper-arm, forearm, and hand motion data collection. The recorded files were processed using OpenSim IMU Placer and IMU Inverse Kinematics. The final demonstration showed that the device can reproduce basic arm movement in OpenSim.

The main contribution of this work is a simple, portable, low-cost, and low-power embedded device for OpenSim IMU data collection. The device has the potential to be useful for students, researchers, and future wearable human motion, animal movement monitoring, farm-based sensing, and exoskeleton-related projects.

References

1. M. Al Borno, J. O’Day, V. Ibarra, J. M. Dunne, A. Seth, A. Habib, Z. Ong, J. L. Hicks, S. D. Uhlich, and S. L. Delp, “OpenSense: An open-source toolbox for inertial-measurement-unit-based measurement of lower extremity kinematics over long durations,” *Journal of NeuroEngineering and Rehabilitation*, vol. 19, no. 1, 2022.
2. P. Slade, A. Habib, J. L. Hicks, and S. L. Delp, “An open-source and wearable system for measuring 3D human motion in real time,” *IEEE Transactions on Biomedical Engineering*, vol. 69, no. 2, pp. 678–688, 2022.
3. OpenSenseRT GitHub Repository, “Does not work with current OpenSim,” GitHub issue 8, 2023. Available: <https://github.com/pslade2/RealTimeKin/issues/8>
4. Seed Studio, “Seed Studio XIAO ESP32C6 Getting Started,” 2024. Available: https://wiki.seedstudio.com/xiao_esp32c6_getting_started/